

Addressing Lexical and Semantic Ambiguity in Natural Language Requirements

Fatima ZAIT

University of Constantine 2 - Abdelhamid Mehri

LIRE Laboratory

Constantine, Algeria

fatima.zait@univ-constantine2.dz

Nacereddine ZAROUR

University of Constantine 2 - Abdelhamid Mehri

LIRE Laboratory

Constantine, Algeria

nasro.zarour@univ-constantine2

Abstract—Ambiguity is a critical problem that rears its ugly head in many disciplines including writing, philosophy, law, and of course software engineering, especially requirements engineering. Requirements are typically expressed in a natural language. However, expressions in natural languages are likely to suffer from ambiguities, where a statement can be reasonably interpreted in more than one way and if we know, even then it is difficult to decide which one is correct. This paper presents an approach for ambiguity detection and resolution in natural language requirements as early as possible using natural language processing and semantic web techniques. Hence, we will be able to identify the ambiguous words and provide them all the possible interpretation and clarifying the meaning of the requirements.

Keywords—requirements engineering, natural language, ambiguity, lexical ambiguity, semantic ambiguity, meaning, natural language processing.

I. INTRODUCTION

Requirements engineering (RE) is the process of identifying, determining and understanding the customer needs towards the new system. This process aims to obtain a description of functional requirements (what the program will actually do) and non-functional requirements (how the software will do it).

The identification of requirements has been defined in various ways by various software engineering practitioners. Usually intuitive and informal, and the requirements are almost always written in a natural language (NL) such as English [1]. Using NL makes the documents easy to comprehend [2] and share. Since NL requirements are often defective (risk of misinterpretations and imprecise inferences), we have to face several language-specific problems inherent in NL such as ambiguity.

A major goal of RE is to achieve a common understanding on the set of requirements between all project stakeholders. Ambiguity occurs when a requirement may be understood in two or more different ways due to a lack of precision. Ambiguity has often been considered as a potentially harmful attribute of requirements [3]. Since ambiguity in the requirements can potentially contribute to a deficiency of interpreting customer's desires, and not being able to document those desires, also is considered as a major case of project failure. For example, if the customer's believed meaning of the requirements is different from that of the system's developers and testers, then the system might not be accepted after customer validation. Similarly, if a user manual is written based on a different interpretation of the requirements from that of the developers, the system runs the risk of being erroneously documented.

Requirements engineers have long recognized that ambiguity is an important challenge in specifying and building software systems [4]. It can cause various issues that influence the system, in light of the fact that ambiguity becomes a bug if not found and settled at early stages [5]. Nevertheless, in industrial practice, the vast majority of requirements documents are written in NL [6].

It is strongly believed that ambiguity in requirements should be tackled as early as possible in the creation of the document. In the last decade, many approaches, tools and techniques have been proposed to detect requirements ambiguity in software requirements. These include the use of a manually curated checklist [7]. Usage of linguistic parsers [8]. And supervised machine learning techniques [9]. These works point out ambiguities from software requirements specification (SRS). We focus on raw requirements that have been elicited using NL and that have not been documented into SRS by adopting natural language processing (NLP), the semantic web technique and similarity measures between requirements, which can help to detect and resolve the issue.

The remainder of this paper is organized as follows. Section II introduces some useful background notions. Section III provides an overview on some proposed work related to our approach. Section IV details our proposition, which is an idea for avoiding the ambiguity during the time of requirement elicitation. Finally, Section V concludes the paper and suggests some future work.

II. BACKGROUND

This part offers some relevant concepts concerning ambiguity in RE, which will be used in the rest of the paper. In particular, in section A, The Ambiguity Handbook [6] classified ambiguities, as lexical, syntactic, semantic, and pragmatic ambiguities at the level of written NL requirements. Additionally, it states vagueness and language errors as further sources of problems. In section B we present an overview of word sense disambiguation. In section C, we report the existed approaches for ambiguity resolution.

A. Classification of ambiguity in NL requirements

According to Berry et al. [6], ambiguity in written NL requirements can be classified to the following classes:

- Lexical ambiguity: arises when a word can have multiple unrelated vocabulary meanings. As noted by Gleich et al. [10] cues at the lexical level (i.e., terms) can be found that lead to other classes of ambiguity.
- Syntactic ambiguity: also called structural ambiguity, it arises when a given sequence or phrase have

multiple grammatical structure where each one provides different meanings.

- Semantic ambiguity: a sentence can be translated into more than one logic expression, in its provided context. In general, the incorrect combination and use of quantifiers may lead to semantic ambiguity. Hence, quantifiers in general, and the term all in particular [11], are regarded as cues of semantic ambiguity.
- Pragmatic ambiguity: it occurs when the whole sentence having a several meanings in the context in which it is expressed. In RE, pragmatic ambiguity was mainly studied for the sub-class of the so-called referential ambiguities, table 1 summarize subtype of which ambiguity. These ambiguities occur when readers might disagree on how a pronoun should be interpreted. Ambiguity cues in these cases are pronouns [10].
- language errors are considered by Berry as a separate class of ambiguity.
- Vagueness: an expression is vague if it admits borderline cases, or relative interpretation. Gleich et al. [10] consider as cue of vagueness any adjective and adverb.

TABLE I. TYPES OF AMBIGUITY [6].

Ambiguity type	Subtype	Example
Lexical ambiguity	Homonymy ambiguity	Two different words with same written but different meaning. (e.g., bank can be the bank of a river, or the business establishment that stores moneys and supplies loans)
	Polysemy ambiguity	A word has several related meanings. e.g: "I like this jacket" (can refer individual jacket or all types of jacket)
Syntactic ambiguity	Analytical ambiguity	When the role of the constituents within a phrase or sentence is ambiguous.
	Attachment ambiguity	A particular syntactic constituent of a sentence, such as prepositional phrase or a relative clause, can be legally attached to two parts of a sentence. e.g.: "The police shot the rioters with guns." "Shot with guns" or "rioters are with guns?"
	Coordination ambiguity	When more than one conjunction, and or is used in a sentence or when one conjunction is used with a modifier. e.g.: "I saw (Peter and {Paul} and Mary) saw me."
	Elliptical Ambiguity	When it is not certain whether or not a sentence contains an ellipsis. E.g: "Peter knows a richer man than Paul." Two meanings, "Peter knows a man who is

		richer than Paul is." and "Peter knows a man who is richer than any man Paul knows."
Semantic ambiguity	Scope ambiguity	(e.g., All lights have a switch; one switch to many lights, or one switch to one light?)
Pragmatic ambiguity	Referential ambiguity	These ambiguities occur when readers might disagree on how a pronoun should be interpreted. (e.g., The system sends a message to the receiver. It sends an acknowledge message; it = system or receiver?).
	Deictic ambiguity	When pronouns, time and place adverbs, such as now and here, and other grammatical features, such as tense, have more than one reference point in the context. "Every student thinks she is a genius." Here is "she" including in every?

B. Word Sense Disambiguation

Word Sense Disambiguation (WSD) is the process of identifying the senses of word based on the context in which it occurs. The most appropriate meaning for a word is selected from a set of admissible meaning, usually known as a sense inventory. WSD techniques use the notion of context in order to decide a particular word sense, a context could differ widely across WSD methods. Such as n-grams (i.e., a sequence of n words including the target word), specifically unigrams (n = 1), bigrams (n = 2), and trigrams (n = 3), to a full sentence or paragraph containing the target word.

Navigli [12] categorized main approaches to WSD as: Supervised WSD, which uses machine learning techniques to learn a classifier from labeled training sets. Unsupervised WSD, which based on unlabeled corpora, and do not exploit any manually sense-tagged corpus to provide sensible choice for a word in context, and Knowledge-based approach rely on external knowledge sources, such as machine-readable dictionaries, thesauri, glossaries, ontologies, etc.

C. Approaches to detect and resolve ambiguity

In RE, two classes of approaches to deal with the ambiguities can be distinguished. The first-class ambiguity prevention includes techniques that detect ambiguities in requirements documents and explain them to the user. This approach warrants that the requirements have to be stated formally, i.e. using precise statements that have a unique meaning. The typical approaches to do so use controlled languages, context free grammar, and style guides/sentence pattern. The second-class ambiguity detection and resolution encompasses techniques that attempt to resolve ambiguities. The ambiguities are not prevented outrightly, but are allowed to remain in the initial SRS, however later appropriate techniques viz. Machine learning, knowledge based and ontology are used to detect and resolve them.

III. RELATED WORK

A lot of research has already been conducted in the field of ambiguity in RE. Starting from manual glossaries approach to automatic ontology based approach to reduce

ambiguity from the requirements. In addition, there are a number of different tools developed to detect and resolve ambiguities. Part of these works is focused on the identification of typical ambiguous terms and constructions [13], [6], [14], [10]. Other works address the ambiguities by translating the requirements into formal languages or models [15], [16], [17]. Finally, some works focus on the usage of NL understanding.

An overview of existing approaches is given in [18], [19]. Shah and Jinwala [18] presented a comparative evaluation of approaches based on pre-selected tools (e.g. Stanford Parser), chosen methods (e.g. rule-based, ontology-based, etc.) or the degree of automation. A further comprehensive overview of existing approaches on disambiguation in the requirements domain is given by Bano [20], which focuses on empirical work. However, all of these surveys show that the requirements have been documented for SRS. It means that the requirements expressed in a professional manner. A large number of solutions only concentrate on the detection of ambiguities. Consequently, the resolution remains the task of the stakeholders.

In order to systematize typical ambiguous phrases, Berry et al. introduced the Ambiguity Handbook [6] which provides a large set of examples of typically dangerous words and constructions. Similarly, Gleich in [10] implemented a tool, although performing lexical and syntactic analysis supported by statistical NLP techniques such as part of speech (POS) tagging, only, is able to detect ambiguities on all levels, from lexical to pragmatic. Although not able yet to detect all ambiguities, as listed in the Ambiguity Handbook, the presented tool should not only detect ambiguities, but also explain, for every detected sentence, what is potentially ambiguous in it. This makes the presented tool suitable not only for ambiguity detection, but also for education purposes. The tool is created only to focus on lexical and syntactic ambiguity.

A tool named Systemized Requirements Engineering Environment (SREE) [8] was created to avoid the ambiguity in the Requirement specification. In this tool the whole document regarding the requirement was loaded and the tool identifies defects based on a pre-defined list of dangerous terms, concentrating on achieving a 100% recall rate and a precision rate of as close as possible to 100%. In this tool they are using Wordnet 3.0 to find out the ambiguous terms. This tool also uses the part-of-speech tagger. For documentation the elicitor must keep in mind the various rules designed for elicitation to bring out the best result. The result generated by SREE is the indication of almost all the possible ambiguities present in the document.

Most tools such as SREE require high user interaction to remove the ambiguities for each ambiguous sentence. Are tedious and time consuming where requirements must be rewritten with minor modifications to match the predefined patterns. Furthermore, software requirements are not fixed in nature, the unpredictable change in the requirements may introduce new ambiguities.

The methods presented above cannot perform an analysis without document requirements engineering standards, such as the INCOSE or IREB recommendations. Thus, tools can only apply to requirements documents SRS. However, these contributions are useful for requirements elicitation, i.e., during the first step of requirements engineering processes.

However, there is no study in the literature that focus on raw requirements. In other words, the requirements are not written in a proper requirement specification style. Our research explored a number of dangerous terms to identify ambiguity. Thus, this research anticipates discovering ambiguity in requirements as early as possible, even before the requirements are professionally documented. Consequently, it will lead to a better quality of detecting and resolving ambiguity in requirements elicitation. The table below presents a comparison between some existing tools and the proposed approach.

TABLE II. COMPETITIVE ANALYSIS

Feature Support	Gleich et al [10]	SREE [8]	Sinha et al. [21]	Nigam et al [5]	Our approach
Normalize Requirements	Yes	Yes	No	No	Yes
Resolving coordination ambiguity	Detection only	Detection only	No	Detection only	Yes
Resolving lexical Ambiguity	Detection only	No	Yes	Detection only	Yes
Resolving semantic ambiguity	Detection only	Detection only	No	No	Yes
User Interaction	High	High	Medium	Low	Low

IV. OUR PROPOSITION

This section covers our approach to detect and resolve ambiguity in raw requirements. The input to our approach is a set of elicited requirements tolerating some degree of ambiguity in natural language. The output is the requirements document without ambiguity. The approach consists of three major steps, as shown in Figure 1. The first step is the pre-processing of the requirement document involves POS tagging and normalization. The second step is the detection of ambiguous words and sentences with its possible interpretation. The third step performs to disambiguate ambiguous words and sentences.

To address the first requirement, we employ lexical databases that store several distinct word senses to determine context knowledge of requirements. WordNet and BabelNet are examples of such lexical databases.

The proposed approach uses Stanford parser [22] for lexical-syntactic analysis and BabelNet [23] for getting the Polysemy words and their synonyms. We choose use BabelNet because it includes encyclopedic knowledge of Wikipedia and further enriches WordNet with contemporary senses.

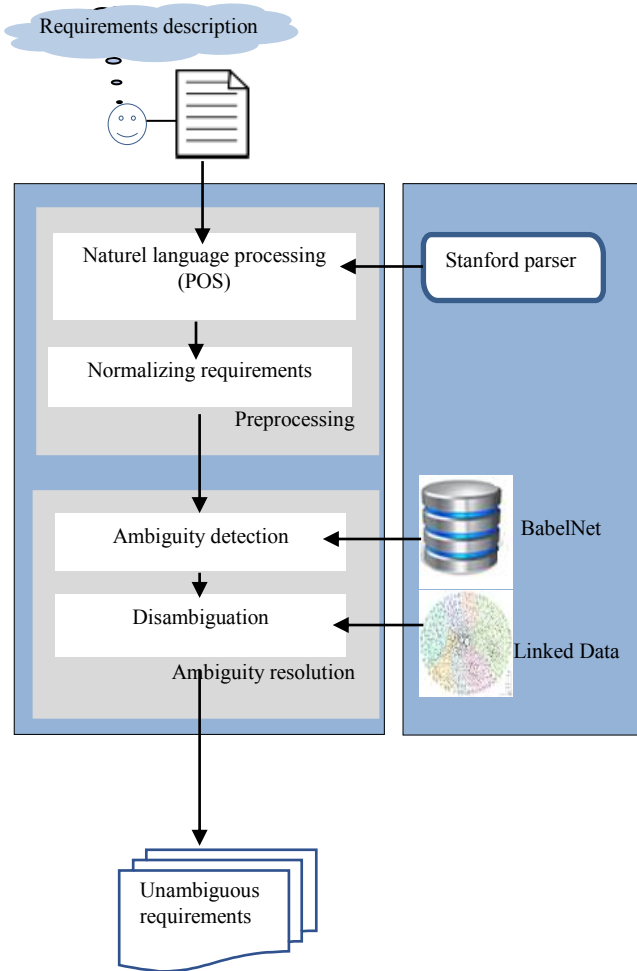


Fig. 1. Overall approach architecture

A. Text pre-processing step

The input raw requirements are split into separate sentences with the end-of-sentence (EOS) detection. To that end, we use the LingPipe Toolkit ¹.

1) *POS tagging*: POS tagging involve the process of dividing the sentence in to noun, verb, pronoun, adverb, helping-verb, adjective, prepositions, etc. For each sentence, the Stanford parser is used to obtain POS tags.

e.g.: “The system sends a message to the receiver.”

The/DT system/NN sends/VBZ a/DT message/NN to/TO the/DT receiver/NN ./.

Dependencies generated by Stanford parser are binary relation used in all modules of the system to identify the relations between two words of a sentence. Table below (table 3) shows some abbreviations used by the Stanford Parser to denote grammatical relationships

TABLE III. SHOWS SOME ABBREVIATIONS USED BY THE STANFORD PARSER TO DENOTE GRAMMATICAL RELATIONSHIPS

Grammatical relations	Explication
Det	determiner
Nsubj	nominal subject
Root	root
Dobj	direct object
Case	adjectival modifier
Nmod	nominal modifier

det(system-2, The-1)
 nsubj(sends-3, system-2)
 root(ROOT-0, sends-3)
 det(message-5, a-4)
 dobj(sends-3, message-5)
 case(receiver-8, to-6)
 det(receiver-8, the-7)
 nmod(sends-3, receiver-8)

2) *Normalizing requirements*: Requirements in natural language contain many compound sentences, connected by connectives ‘and’ or ‘or’. The second phase is the normalization of compound sentence with an algorithm using rules defined in [9].

E.g.: “Binary kernels provide efficient storage and access to large datasets.”

The above example normalized as:

- Binary kernels provide efficient storage.
- Binary kernels provide efficient access to large datasets.

B. Ambiguity resolution

This process consists of two main steps: Step I is to detect ambiguous sentences, Step II is a disambiguation function which assigns at least one sense taken from the set of all denotations to the given ambiguous requirements.

1) *Ambiguous Instance Detection step*: Ambiguities should be identified in raw requirements. This step involves the process of classifying ambiguity as Lexical and semantic, identifies a set of possible candidates meaning for each identified ambiguous requirement. See figure 2.

a) *Ambiguity indicators detection*: Works on following steps:

1. Take normalized requirements as input.
2. Then each word in that requirement will be used to find the ambiguous words (key words) with the help of BabelNet.
3. If the key words interpretation is more than one then that word will be listed below with all the possible interpretations having same Part of

¹ <http://www.alias-i.com/lingpipe/>

Speech of that word and mark it as lexical ambiguity.

4. For each requirement, not marked as lexical ambiguity, match all words against the black list. If word is matched then mark the sentence as semantic ambiguity.
5. Calculate the total number of lexical and semantic ambiguities.
6. Calculate the percentage of ambiguities detected.

C. Ambiguity classification

The process of classifying ambiguity as lexical and semantic displays detected ambiguous word in colored form. The ambiguous word in lexical ambiguity will be colored as green and the semantic as red.

D. Candidates generation

- Lexical ambiguity: Apply n gram approach (to use context of the word).

The knowledge of the context in which the word is used is very important to better understand the most accurate meaning (sense) of a lexical ambiguous word. For this we use n-gram approach that provides a contiguous sequence of n items (words) from a given sentence. Formally, an n-gram is a consecutive subsequence of length n of some sequence of tokens $w_1 \dots w_n$

Eg: The **device** must perform the safety procedure when the temperature reading is greater than 150 °

The word device may have fifteen possible meanings in five contexts:

- Engineering and technology (machine, mobile phone...)
- Computing (hardware, computer hardware...)
-

In this step we use the neighborhood noun unambiguous to determine the context of the requirement. For example, the word procedure is considered unambiguous word (token $n^{\circ}:07 \Rightarrow n=7$), so that the processor looks up the context knowledge to find device senses labeled with procedure context. According to the above meaning, only senses labeled with procedure context will be considered.

- Semantic ambiguity: can be the case, for example, when several quantifiers occur in the same sentence, contains the indicators: all, any, few, little, many, much, several, and some. Grouped in black list.

like in “*all* citizens have *a* social security number” that can be interpreted in two ways:

- every citizen has an individual social security number:

$\forall x.\text{citizen}(x) \Rightarrow \exists y.\text{social_security_number}(y) \wedge \text{has}(x, y)$

- all citizens have the same social security number:

$\exists y.(\text{social_security_number}(y) \forall x.(\text{citizen}(x) \Rightarrow \text{has}(x, y)))$

The output of this step will serve as an input to the disambiguation step.

2) *Disambiguation step*: Our disambiguation method is a Graph-based Centrality for WSD. Given a sequence of words with their corresponding admissible meaning, the disambiguation algorithm seeks to identify a graph of concepts dependencies on which the centrality can be measured, resulting in a set of scores that can be used for meaning assignment. The algorithm consists of three main steps:

1. Construction of meaning dependencies graph where $G=\{S(D), R(D), sf\}$

$S(D)=\{s_1, s_2, \dots, s_n\}$, is the set of meaning.

$R(D)=\{r_1, r_2, \dots, r_m\}$, dependencies between pairs of meaning (s) are represented as directed or undirected edges.

sf is a scoring function.

2. Meaning scoring using graph-based centrality algorithms;
3. Meaning assignment.

In this step the concepts generated in the above step will be examined by the linked data based on its relations that extracted from the requirement. For example, perform is identified as a relation between the node device and the node procedure. This simple graph is then mapped to the linked data. The output of these processing steps will be unambiguous.

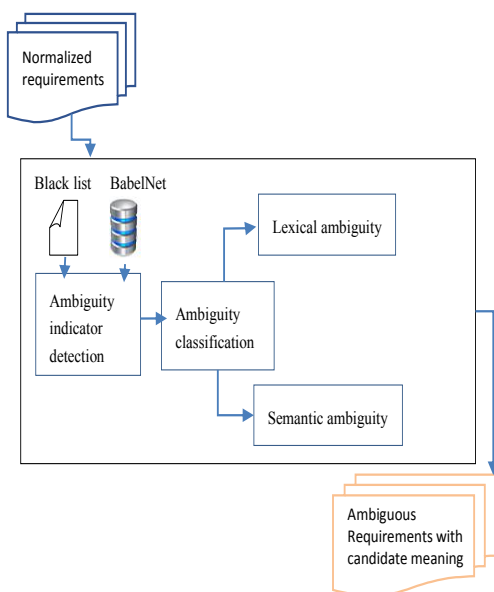


Fig. 2. Ambiguity detection

V. THE EXPERIMENT

The approach consists of three major steps, as shown in Figure 1: (1) pre-processing of the requirement document involves POS tagging and normalization, (2) the detection of ambiguous words and sentences with its possible interpretation, and (3) disambiguate ambiguous words and sentences.

Data selected for this research is an open source data, which is a case study project of University College London (UCL) [24]. The data gathered in the project is RALIC project (Replacement Access, Library and ID Card). It is a software project to enhance the existing access control system at UCL.

RALIC aims to substitute the outdated access control systems, combining various existing access control mechanisms, and at the minimal, combine the photo ID card, access card, and library card. RALIC is a combination of development and customization of an off-the-shelf system [24]. The scope of the project is presented in Table 4.

Requirements elicitation is the activity in which stakeholder needs are understood. Due to the variety of stakeholders, requirements gathered for RALIC rather have some ambiguous. requirements. For example, members of the UCL Development and Corporate Communications Office preferred the ID card to have UCL branding, but the security guards prefer otherwise for security reasons in case the cards are lost.

TABLE IV. RALIC PROJECT SCOPE.

Scope Item	Description
1	Replace swipe card readers with smart card readers
2	Source and install access card printers
3	Decide on card design and categories
4	Define user groups and default access rights Provide a more accurate card holder database, save resources on manual data input, and facilitate automated
5	provision and suspension of access and library borrowing rights
6	Issue new cards to staff, students, visitors and contractors
7	Replace the Library access control system
8	Use new cards at the Bloomsbury Fitness Centre

TABLE V. EXERPT OF RALIC ELICITED REQUIREMENTS

No	requirements
1	User friendly system which does not require a complex training programme
2	The system must be secure with clear procedures for dealing with ID
3	System is maintainable and supported by suppliers for a minimum of 5 years
4	System is able to continue operation of up-to-date hardware and server operating systems
5	The cards should have a photo and name on the front/cover
6	The card can be extended for future requirements, such as a digital certificate
7	The system should be easy to maintain and support
8	all software including packages and all oracle objects
	

For finding ambiguities, all sentences are tagged one by one according to parts of speech, normalized the requirement and mapped to BabelNet and black list to find ambiguous word. As shown in the above sentences that includes such as, user friendly, simple, minimum, and easy. When these vague words are found in requirements, it can cause the requirements to be ambiguous.

The implementation of the approach to detect ambiguities in elicited requirements i.e.: Lexical and semantic ambiguities. Figure 3 shows the interface of the implemented tool. It consists of three min windows.

A. Editor window

Where users can directly write the requirements and processed them line by line for ambiguity detection

B. Textual result window

This window highlights ambiguous word in requirements document, in color form (green for lexical ambiguity, red for semantic ambiguity). Which will be displayed after clicking the detection button.

C. Graphical result window

This window shows the different meaning of ambiguity detected where the user displays the cursor on ambiguous requirements. And shows the total ambiguities and the percentage of each ambiguity type in a graphical representation.

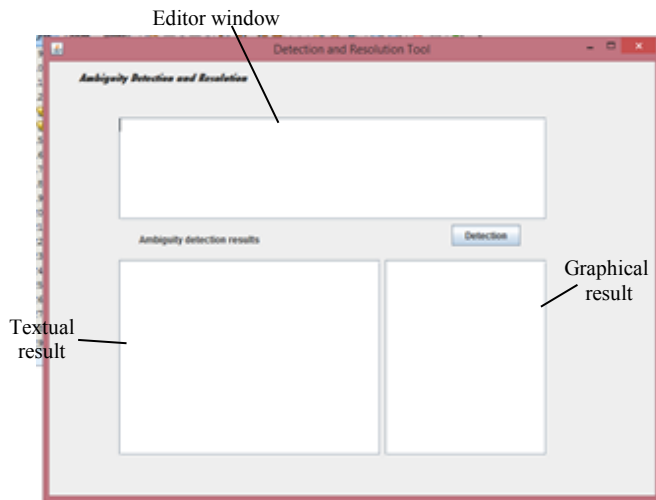


Fig. 3. Interface of ambiguity detection and resolution tool

Natural language is initially used to write the user requirements. We need ways to cope with the ambiguity inherent in natural language. Thus, in this paper we described an automated approach for detecting and resolving lexical and semantic ambiguity

VI. CONCLUSION AND FUTURE WORK

At the first stage of software development, requirements define what future users want the system to do. The requirements in general are expressed in natural languages which are inherently ambiguous. Moreover, we know that Ambiguity is the biggest problem in requirement gathering which may lead to failure of software if the requirements are not properly interpreted or SRS is not properly designed because the rest of the project depends upon this initial step. We need ways to deal with the ambiguity inherent in NL.

In this paper, we addressed the problem of automatically identifying and resolving lexical and semantic ambiguities in raw requirements, in order to minimize its effects. The application of POS tagger can help to find syntactically ambiguous concepts in user requirements and the use of linked data for disambiguate semantic ambiguity.

This study is performed in the early stage. In future vagueness ambiguity, incompleteness, referential, and pragmatic ambiguity can also be considered. Also, project to build an ontology for requirements elicitation.

REFERENCES

- [1] K. Pohl, Requirements engineering: fundamentals, principles, and techniques, 1st ed, Springer, 2010.
- [2] C. Arora, M. Sabetzadeh, L. Briand, and F. Zimmer, "Automated checking of conformance to requirements templates using natural language processing", IEEE Transactions on Software Engineering vol. 41(10), pp. 944–968, 2015.
- [3] S. Boyd, D. Zowghi, and A. Farroukh, "Measuring the expressiveness of a constrained natural language: An empirical study," In: Proceedings of the 13th IEEE International Conference on Requirements Engineering (RE'05), Washington, 2005, pp. 339–352.
- [4] G. Roman, "A taxonomy of current issues in requirements engineering", in Computer, vol. 18(4), pp. 14–23, 1985.
- [5] A. Nigam, N. Arya, B. Nigam, and D. Jain, "Tool for automatic discovery of ambiguity in requirements", IJCSI International Journal of Computer Science Issues, vol. 9(5), pp. 350–356, 2012.
- [6] D. M. Berry, E. Kamsties, and M. M. Krieger, "From contract drafting to software specification: Linguistic sources of ambiguity", 2003.
- [7] P. Jain, K. Verma, A. Kass, and R.G. Vasquez, "Automated review of natural language requirements documents: generating useful warnings with user-extensible glossaries driving a simple state machine", In: India SE conference, ACM, India, pp. 37–46, 2009.
- [8] S. Tjong, D. Berry, "The design of sree – a prototype potential ambiguity finder for requirements specifications and lessons learned", In: Proceedings of REFSQ, pp. 80–95, 2013.
- [9] H. Yang, A. Willis, A.D. Roeck, and b. Nuseibeh, "Automatic detection of noxious coordination ambiguities in natural language requirements", international conference on automated software engineering, IEEE/ACM, 2010.
- [10] B. Gleich, O. Creighton, and L. Kof, "Ambiguity detection: Towards a tool explaining ambiguity sources" In: REFSQ 10, LNCS, vol. 6182, pp.218–232. Springer, 2010.
- [11] C. Rupp, R. Goetz, "Linguistic methods of requirements-engineering (NLP)", in EuroSPI 00, 2000.
- [12] R. Navigli, "Word sense disambiguation: A survey" ACM Computational Surveys, vol. 41(2), 2009.
- [13] D. M. Berry, E. and E. Kamsties, "The syntactically dangerous all and plural in specifications. IEEE Software, vol. 22(1), pp. 55–57, 2005.
- [14] S. Gnesi, G. Lami, and G. Trentanni, "An automatic tool for the analysis of natural language requirements", IJCSSE, vol. 20(1), 2005.
- [15] V. Ambriola, and V. Gervasi, "On the systematic analysis of natural language requirements with Circe", ASE, vol. 13, 2006.
- [16] L. Kof, "From requirements documents to system models: A tool for interactive semi-automatic translation", In: RE'10, 2010.
- [17] A. Cimatti, M. Roveri, A. Susi, and S. Tonetta, "Formalizing requirements with object models and temporal constraints. SoSyM" vol. 10(2), 2011.
- [18] U. S.Shah, and D. C.Jinwala, "Resolving ambiguities in natural language software requirements: A comprehensive survey. ACM SIGSOFT Software Engineering Notes, vol. 40, pp. 1-7, 2015.
- [19] S. Husain, and R. Beg, "Advances in ambiguity less NL SRS: A review" In: Proceedings of ICETECH, pp. 221–225. India 2015.
- [20] M. Bano, "Addressing the challenges of requirements ambiguity: A review of empirical literature", In Proceedings of the 5th International Workshop on EmpiRE, pp. 21–24. IEEE, (Ottawa, ON, Canada), 2015.
- [21] S. Sinha, and M. S. Husain "Proposal for avoiding ambiguity in requirement engineering using artificial intelligence", 2016.
- [22] <https://nlp.stanford.edu/software/>, last accessed 2018/02/28.
- [23] www.babelnet.org, last accessed, last accessed 2018.
- [24] S. L. Lim, "Social networks and collaborative filtering for large-scale requirements elicitation", Doctoral dissertation, University of New South Wales, 2010